

Can Learned Signals Predict and Eventually Guide Combinatorial Search?

Joshua King

Toronto, Ontario, Canada

August 19, 2026

ABSTRACT

This study investigates whether the computational difficulty of reconstructing a permutation matrix from partial invariant information can be predicted using conventional regression methods. Permutation matrices were generated by reshaping random permutations of $(1, \dots, 25)$ into (5×5) matrices, after which subsets of row, column, diagonal, and antidiagonal sum and product invariants were independently observed. Each resulting constraint satisfaction problem was solved using Google OR-Tools CP-SAT, and solver difficulty was measured using the logarithm of one plus the number of search branches explored. A dataset of 10,000 simulated reconstruction instances was used with a test-train split of 80-20 to compare linear regression, ridge regression, and regression trees using a common predictive evaluation framework. The invariant feature representation contained 24 predictors consisting of observed sums and log-transformed products, with unobserved invariants encoded as zero. The results showed that linear regression and ridge regression achieved similar predictive performance, explaining approximately 33% of the variation in log solver branches, whereas the regression tree performed noticeably worse. These findings demonstrate that partial invariant information contains meaningful predictive signal while also suggesting that conventional regression models struggle to fully capture the combinatorial structure that may dictate reconstruction difficulty. Future work should investigate structured representations, including graph neural networks and explicit missingness indicators, to encode the relational dependencies inherent to permutation reconstruction.

Keywords: Combinatorial Search, Constraint Programming, Learned Search Heuristics

RESEARCH QUESTION

Can statistical models learn useful signals about the computational difficulty of an exact combinatorial reconstruction problem, and can those signals eventually be used to guide the search itself?

PROBLEM

Consider an $(n \times n)$ matrix containing every integer from (1) to (n^2) exactly once. Instead of observing the matrix directly, we observe permutation-invariant quantities such row and column sums and logarithms of row/column products. The reconstruction problem is deceptively simple to state: recover a valid arrangement consistent with the observations. However, the underlying search space grows factorially, while multiple valid reconstructions can exist for some observation sets.

Example

The below observation matrices each correspond to the same permutation of size nine. Try to replace each * in each matrix with a unique number between 1 and 9 inclusive. Note that log-sums are presented in their exponential form ($e^{\log\text{-sum}}$) and are underlined for convenience of expression. Sums are *italicized*

for convenience of reference.

$$\begin{bmatrix} - & - & 11 & - & - \\ 16 & * & * & * & - \\ - & * & * & * & 40 \\ - & * & * & * & - \\ - & - & - & 63 & - \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} - & - & - & 17 & - \\ 16 & * & * & * & - \\ - & * & * & * & - \\ - & * & * & * & 84 \\ - & 144 & - & - & - \end{bmatrix}$$

This creates a useful experimental setting in which **the solution is exactly verifiable while the underlying search problem can be extremely large.**

APPROACH

I generated permutation matrices and independently sampled subsets of invariant observations. Each instance was passed to an exact constraint-programming solver, providing a computer-verifiable reconstruction and measurements of the computational search required to obtain it. I then constructed statistical features from the observed invariants and trained regression models to predict solver behaviour. The initial models include:

1. Linear Regression
2. Ridge Regression
3. Regression Tree

Target variables include solver-derived quantities such as branches, conflicts, and propagations.

Initial Result

On a dataset of **10,000 generated instances**, the predictive models captured a minority portion of the variation in reconstruction outcomes.

For the primary regression target:

Model	MAE	RMSE	R ²
Linear	1.5589	2.1989	0.3306
Ridge	1.5624	2.1989	0.3306
Tree	1.3872	2.2859	0.2763

Table 1. Model results evaluated on 2,000 random instances.

The results suggest that **observable structural information contains predictive information about the difficulty of an otherwise combinatorial search problem.** Changing the origination seed alters these error values suggesting the models are learning observation patterns from the particular generated training instances rather than learning genuine structural difficulty. A possible explanation for the high rate of generalization error is that the relatively small samples used to train the models are not reflective of the overall population dynamics. The population size grows at a super-factorial rate, which small samples fail to account for. This idea suggests a shift from raw prediction, to informed search.

WHY THIS MATTERS

The current experiment treats the learned model only as a predictor. The more interesting question is whether the prediction can become part of the solver. This would turn the model from a passive difficulty estimator into a component of an adaptive search strategy.

observations → learned search signal → search decisions → verified solution

FOLLOW-UP QUESTIONS

1. Can simulated annealing provide improved performance over predictive models?
2. Can loss decomposition induce a compatibility measure that improves model performance?
3. Can learned heuristics reduce branches/conflicts without sacrificing exactness?
4. Does the approach transfer to larger matrices or other invariant reconstruction problems?
5. Can reinforcement learning learn useful search operators on top of the exact solver?

KEY IDEA

The broader motivation is to study a setting where **learning and exact search can be combined without making the final answer unverifiable**: learning proposes where to look and the combinatorial solver determines whether the proposed solution is actually valid.

Status: Computational experiment / preliminary result

Code: https://github.com/algorithmic-andy/permutation_reconstruction

Last updated: September 2026

Contact: joshking194@hotmail.com · <https://www.linkedin.com/in/josh-king-in/>

Full text available upon request